Object–oriented programming

Instructor: Masoud Asghari

Web page: http://www.masses.ir/lectures/oops2017sut

Ch: 1

In this slide

- We follow:
  - https://docs.oracle.com/javase/tutorial/index.html

Trails Covering the Basics

These trails are available in book form as *The Java Tutorial, Sixth Edition*. To buy this book, refer to the box to the right.

- Getting Started — An introduction to Java technology and lessons on installing Java development software and using it to create a simple program.
- Learning the Java Language — Lessons describing the essential concepts and features of the Java Programming Language.
- Essential Java Classes — Lessons on exceptions, basic input/output, concurrency, regular expressions, and the platform environment.
- Collections — Lessons on using and extending the Java Collections Framework.
- Date-Time APIs — How to use the `java.time` pages to write date and time code.
- Deployment — How to package applications and applets using JAR files, and deploy them using Java Web Start and Java Plug-in.
- Preparation for Java Programming Language Certification — List of available training and tutorial resources.

Installing JAVA *JDK* and IDE from http://www.oracle.com/technetwork/java/javase/downloads/index.html

Java Platform, Standard Edition

Java SE 8u60

JDK
DOWNLOAD

Server JRE
DOWNLOAD

JRE
DOWNLOAD

Java SE Development Kit (JDK) Cobundles

JDK 8 with NetBeans
This distribution of the JDK includes the NetBeans IDE, which is a powerful integrated development environment for developing applications on the Java platform. Learn more

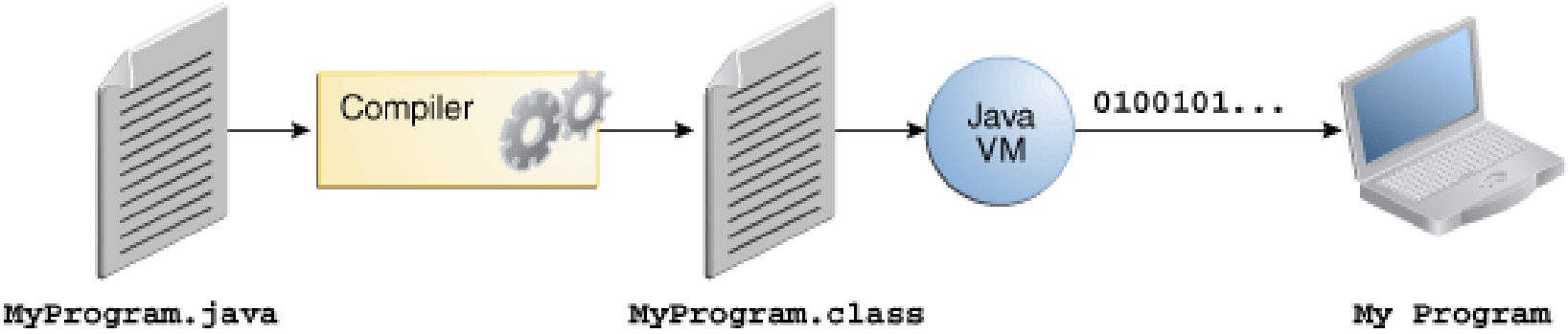
DOWNLOAD

Which Java package do I need?

- **Software Developers: JDK** (Java SE Development Kit). For Java Developers. Includes a complete JRE plus tools for developing, debugging, and monitoring Java applications.
- **Administrators running applications on a server: Server JRE** (Server Java Runtime Environment) For deploying Java applications on servers. Includes tools for JVM monitoring and tools commonly required for server applications, but does not include browser integration (the Java plug-in), auto-update, nor an installer. Learn more
- **End user running Java on a desktop: JRE:** (Java Runtime Environment). Covers most end-users needs. Contains everything required to run Java applications on your system.

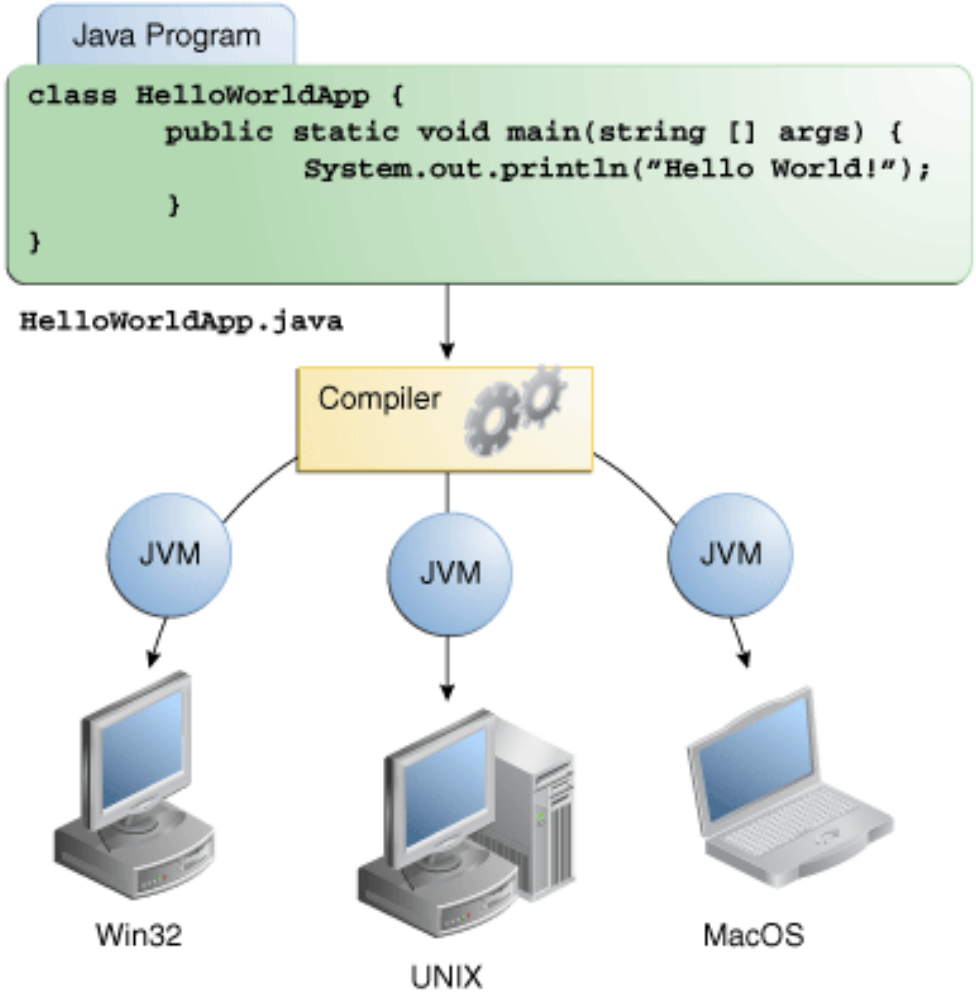
Trail: Getting Started > The Java Technology Phenomenon

- Java technology is both a programming language and a platform.
- In the Java programming language,
  - All **source code** is first written in **plain text** files ending with the .java extension.
  - Those source files are then **compiled** into .class files by the javac compiler.
    - A .class file does not contain code that is native to your processor;
    - it contains bytecodes (the machine language of the Java Virtual Machine (JVM).
  - The java launcher tool then runs your application with an instance of the Java Virtual Machine.



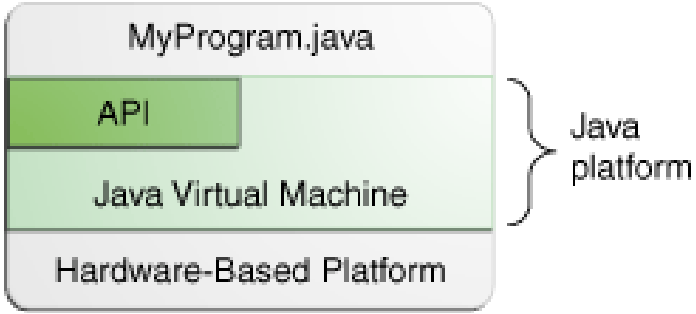
Trail: Getting Started > The Java Technology Phenomenon

- Because the Java VM is available on many different operating systems,
  - The same .class files are capable of running on
    - Microsoft Windows, Linux, or Mac OS.



Trail: Getting Started > The Java Technology Phenomenon

- **The Java Platform**
  - A platform is the hardware or software environment in which a program runs.
  - Most platforms can be described as a combination of the operating system and underlying hardware.
- The **Java platform** has two components:
  - The **Java Virtual Machine**
    - it's the base for the Java platform
    - and is ported onto various hardware-based platforms
  - The Java Application Programming Interface (**API**)
    - The API is a large collection of ready-made software components that provide many useful capabilities.
    - It is grouped into <u>libraries</u> of related classes and interfaces;
    - these libraries are known as <u>packages</u>.
- As a platform-independent environment, the Java platform can be slower than native code.

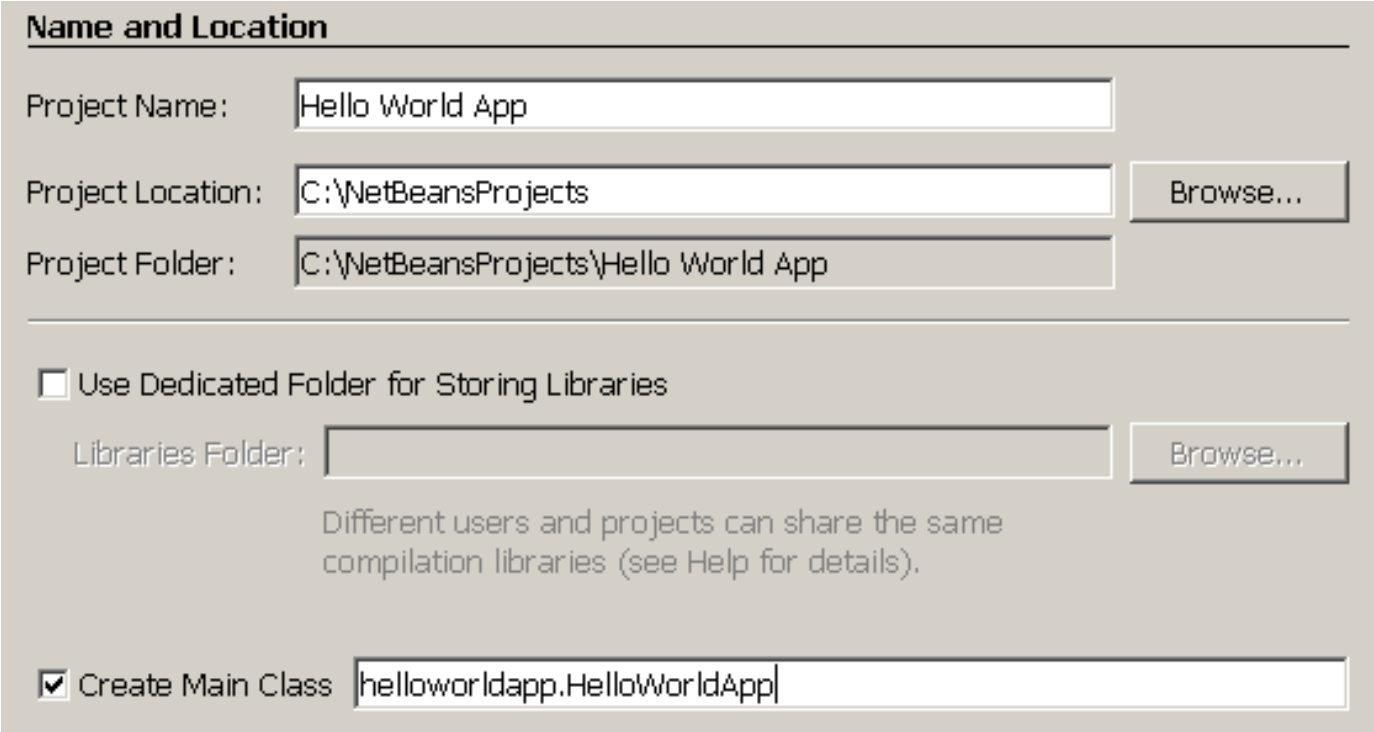


Trail: Getting Started > The "Hello World!" Application

- "Hello World!" for the NetBeans IDE
- **Creating Your First Application**
- To create this program, you will:
  1. **Create an IDE project**
     - When you create an IDE project, you create an environment in which to build and run your applications.
     - You can build or run your application by choosing a single menu item within the IDE.
  2. **Add code to the generated source file**
     - As part of creating an IDE project, a skeleton source file will be automatically generated.
     - You will then modify the source file to add the "Hello World!" message.
  3. **Compile the source file into a .class file**
     - The IDE invokes the Java programming language compiler (javac),
       - which takes your source file and translates its text into instructions that the Java virtual machine can understand.
       - The instructions contained within this file are known as bytecodes.
  4. **Run the program**
     - The IDE invokes the Java application launcher tool (java),
       - which uses the Java virtual machine to run your application.

Trail: Getting Started > The "Hello World!" Application

- **1. Create an IDE Project**
  - Launch the NetBeans IDE.
  - In the NetBeans IDE, choose **File | New Project...**.
  - In the **New Project** wizard, expand the **Java** category and select **Java Application**
  - In the **Name and Location** page of the wizard, do the following:
    - In the **Project Name** field, type Hello World App.
    - In the **Create Main Class** field, type helloworldapp.HelloWorldApp.



Trail: Getting Started > The "Hello World!" Application

Hello World App - NetBeans IDE 8.0

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help Search (Ctrl+I)

<default config>

Projects Files Services

- Hello World App
  - Source Packages
    - helloworldapp
      - HelloWorldApp.java
  - Libraries

HelloWorldApp - Navigator

Members <empt...

- HelloWorldApp
  - main(String[] args)

HelloWorldApp.java

Source History

```
1  /*
2   * To change this license header, choose License Headers i
3   * To change this template file, choose Tools | Templates
4   * and open the template in the editor.
5   */
6
7  package helloworldapp;
8
9  /**
10  *
11  * @author
12  */
13 public class HelloWorldApp {
14
15     /**
16      * @param args the command line arguments
17      */
18     public static void main(String[] args) {
19         // TODO code application logic here
20     }
21
22 }
23
```

11:11 INS

Trail: Getting Started > The "Hello World!" Application

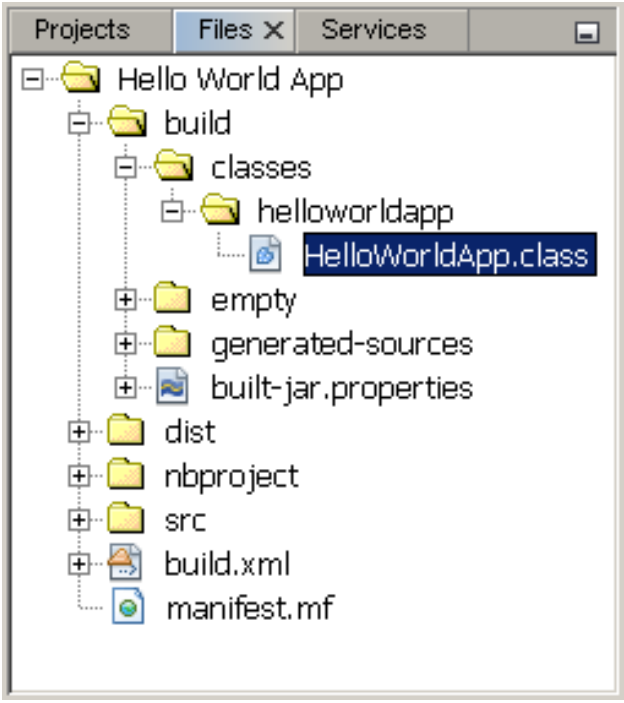
- **2. Add Code to the Generated Source File**
  - When you created this project, you selected the Create Main Class checkbox
  - The IDE has therefore created a **skeleton class** for you.
  - You can add the "Hello World!" message to the skeleton code by replacing the line:
    - // TODO code application logic here
  - with the line:
    - System.out.println("Hello World!");
  - Save your changes by choosing **File** | **Save**.

```
package helloworldapp;

public class HelloWorldApp {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

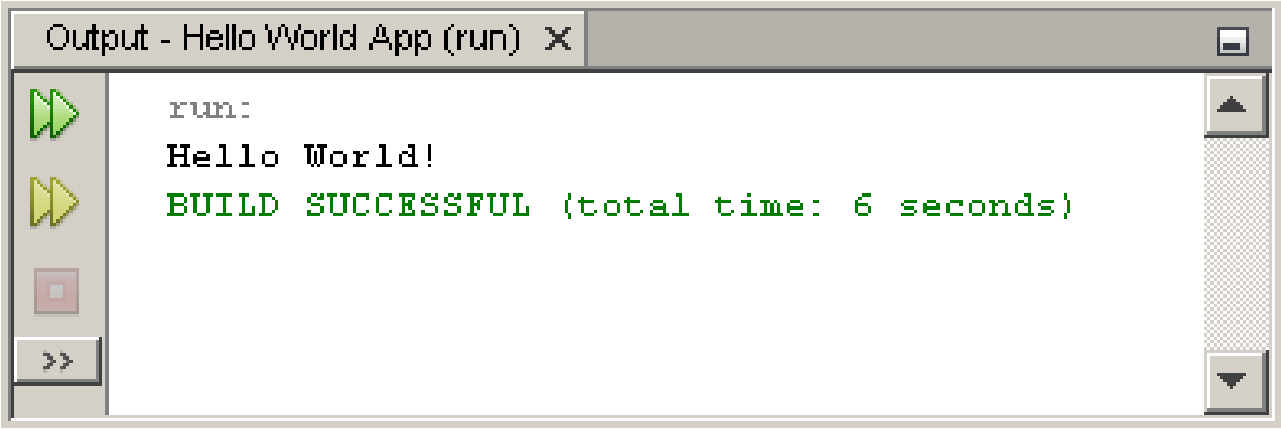
Trail: Getting Started > The "Hello World!" Application

- **3. Compile the Source File into a .class File**
  - To compile your source file, choose **Run** | **Build Project (Hello World App)** from the IDE's main menu.
    - If the build output concludes with the statement BUILD SUCCESSFUL,
      - You have successfully compiled your program!
    - If the build output concludes with the statement BUILD FAILED,
      - you probably have a syntax error in your code.
      - Errors are reported in the Output window as hyperlinked text.
      - You double-click such a hyperlink to navigate to the source of an error.
  - When you build the project, the bytecode file HelloWorldApp.class is generated.



Trail: Getting Started > The "Hello World!" Application

- **4. Run the Program**
  - From the IDE's menu bar, choose **Run** | **Run Main Project**.



Trail: Getting Started > The "Hello World!" Application

- Some IDE behavior that you are likely to see
  - Once you have created a project in the IDE, you can add files to the project using the New File wizard.
    - Choose **File | New File**, and then select a template in the wizard, such as the Empty Java File template.
  - You can compile and run an individual file (as opposed to a whole project) using the IDE's **Compile File** (F9) and **Run File** (Shift-F6) commands.
    - If you use the **Run Main Project** command, the IDE will run the file that the IDE associates as the main class of the main project.
  - As you are typing in the IDE,
    - a code completion box might periodically appear.

Trail: Getting Started > A Closer Look at the "Hello World!" Application

```
public class HelloWorldApp {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

- The keyword class
  - begins the class definition followed by a class name, and the code for each class appears between the { }
- In the Java, every application must contain a main method
  - whose signature is:
    - public static void main(String[] args)
  - The main method is similar to the main function in C and C++; it's the entry point for your application
  - The main method accepts a single argument: an array of elements of type String.
    - This array is the mechanism for runtime system to pass information to your application.
    - For example: java MyApp arg1 arg2
- the line: System.out.println("Hello World!");
  - uses the System class from the core library (API) to print the "Hello World!" message to standard output.